UNIVERSITETET
I OSLO



SoppSporer

Prosjektoppgave i IN2000

Case 5: Felles datakatalog

Team 16

Adam Nordli Ben Nasr	- adamnb
Esma Bouhaddouz	- esmabo
Jostein Ibsen	- josteihi
Mikael Fossbakk	- mikaefo
Harriet Mellquist	- hamellqu
Heidi Lilledal Østmoe	- heidilos

Veiledere:

Sophie Isabelle Vonholm og Sindre Rolstad Denby

UIO - Informatikk

Innholdsfortegnelse

Innholdsfortegnelse **1**
Oversikt over vedlegg **2**
Presentasjon **3**
Presentasjon av caset og prosjektet 3
Kort beskrivelse av løsningen 3
Møt teamet 4
Brukerdokumentasjon **5**
Målgruppe 5
Funksjonalitet 5
Plattformer og aksessering 7
Produktdokumentasjon **8**
Kvalitetsegenskaper ved appen 8
Bruk av API-er 12
Prosessdokumentasjon **14**
Utviklingsmetode og bruk av smidige praksiser 14
Oversikt over prosjektets gang 16
Endringer i kravspesifikasjonen 18
Testing av løsningen underveis 19
Refleksjon **21**
Tverrfaglig arbeid 21
Smidig utviklingsmetode 22
Utfordringer 24
Avslutning **25**
Referanseliste **27**



Oversikt over vedlegg

1. Tilbakemelding til MET

Inneholder vår vurdering av Felles datakatalog og GeoNorge, som beskrevet i oppgaveteksten til case 5.

2. Funksjonelle og ikke-funksjonelle krav

Inneholder en oversikt over appens funksjonelle og ikke-funksjonelle krav, ved prosjektstart og prosjektslutt.

3. Kode utenfor appen

Inneholder en kort oppsummering og utdrag fra kode som har blitt skrevet for å hente inn og lagre oversikter over værstasjoner, artsfunn og lagringen av maskinlæringsmodellen. Dette er kode som ikke er direkte en del av appen, men relevant.

4. Affinity-digram

Inneholder et Affinity-diagram laget med utgangspunkt i fem semi-strukturerte intervjuer.

5. Wireframes

Inneholder wireframes laget i Figma og brukt i brukertesting.

6. Resultat fra spørreundersøkelse

Inneholder samlet resultat fra spørreundersøkelse.

7. Prosseslogg

Inneholder vår prosesslogg hvor vi redegjør for alle teamaktivitetene vi har gjennomført i løpet av prosjektet. Merk: Ligger i egen innleveringsmappe.





Presentasjon

I denne rapporten vil vi presentere prosjektarbeidet og løsningen vår i emnet IN2000. Gjennom brukerdokumentasjonen ønsker vi å formidle appens funksjonalitet. Videre vil vi i produktdokumentasjonen beskrive appens kvalitetsegenskaper og hvilke APIer vi har brukt i løsningen. Vi vil også redegjøre for brukerundersøkelsene vi har holdt underveis og hvordan vi har utført datainnsamling og analyse. I prosessdokumentasjonen går vi gjennom våre smidige praksiser, gir en oversikt over prosjektets gang og beskriver hvordan løsningen vår har blitt testet underveis i prosjektet. I siste del av rapporten reflekterer vi over hvordan vi har jobbet tverrfaglig i teamet, effekten av smidig arbeid og utfordringer vi har opplevd underveis i utviklingsprosessen. Som en avslutning gir vi våre beste tips til neste års studenter.

Presentasjon av caset og prosjektet

Teamet valgte Case 5: Åpent case med de felles datakatalogene data.norge.no fra DigDir og GeoNorge fra Kartverket. Appen vår SoppSporer er en posisjonsbasert app for Android som estimerer muligheten for å finne utvalgte sopptyper på en valgt lokasjon i Norge. Appen kombinerer data fra en rekke APIer fra casets datakaloger, deriblant biologiske preferanser for art og geografiske data, med værdata fra MET. Den innsamlede dataen brukes til å beregne en score på 0-100 som formidler muligheten for soppfunn i sanntid.

Kort beskrivelse av løsningen

I SoppSporer kan brukeren lete etter muligheter for soppvekst i hele Norge. Ved å velge seg en sopp fra de 11 utvalgte sopp-profilene appen tilbyr og deretter trykke i kartet, vil man få presentert en score som forteller om mulighetene for soppfunn på valgt punkt i sanntid. Det finnes også flere tilleggsfunksjoner som kan forenkle letejobben; henholdsvis heatmap, skogstype-visualisering og visning av funn registrert i Artsdatabanken. Vår visjon med appen er at den både skal gjøre det lettere for allerede aktive og erfarne sopplukkere å finne nye soppsteder, men også at den skal hjelpe nybegynnere å komme i gang med sopplukking. Vi ønsket en visuell profil som minnet om klassiske soppbøker, ettersom det kom frem i innsiktsarbeid at dette er et verktøy mange sopplukkere allerede bruker. SoppSporer skiller seg fra lignende apper på markedet ved at den er prediktiv. Der andre apper hovedsakelig viser historiske funn, peker vår app på nye, uoppdagede områder som har gode forhold akkurat nå. Slik kan brukerne spres utover i stedet for å samles på samme koordinat.



Møt teamet

Møt teamet bak prosjektet! Vi er en engasjert gruppe studenter som bringer med oss ulik erfaring fra både akademia, tidligere studier og livserfaringer. Her er en oversikt over hvem vi er og bakgrunnen vår.



Heidi Lilledal Østmoe

Studie: Design, bruk, interaksjon

Styrke: Designfag og overblikk

Heidi er 26 år gammel, og har tidligere studieerfaring fra litteraturvitenskap, historie og filosofi. På fritiden gjør hun mye forskjellig, fra strikking til gaming. Hun har ikke mye erfaring med sopp-plukking, men gleder seg til å teste SoppSporer i høst.



Adam Nordli Ben Nasr

Studie: Programmering og systemarkitektur

Styrke: Bredt interessefelt og godt humør

Adam er 22 år gammel. Han har tidligere erfaring som servitør, bartender og driftsoperatør. På fritiden liker han å være med venner og familie, styrketrening, klatring/buldring og matlaging. Han kan tenke seg å teste SoppSporer i sommer for å finne fersk matsopp.



Jostein Holen Ibsen

Studie: Programmering og systemarkitektur

Styrke: Tverrfagelig kompetanse og erfaring med maskinlæring

Jostein er 26 år gammel og egentlig stipendiat på OUS. Han jobber til vanlig med forskning på diagnostikk og proteomikk for fordøyelsesykdommer



Mikael Fossbakk

Studie: Design, bruk, interaksjon

Styrke: Benkpress og algoritmer.

Mikael er 27 år gammel. Han har erfaringer fra helsefag og sosiologi. Mikael er glad i dyr og setter pris på gode venner og sene kvelder. Han vurderer nå å kanskje plukke sopp i sommer.



Esma Bouhaddouz

Studie: Design, bruk, interaksjon

Styrke: Lange sesjoner med debugging

Esma er 21 år gammel og har erfaring fra kunst og kuratering. På fritiden liker hun å drive med kampsport og lese. Hun har ikke en interesse i å plukke sopp, men vil veldig gjerne teste SoppSporer og samle sopper som pokèmon



Harriet Mellquist

Studie: Programmering og systemarkitektur

Styrke: Struktur og oversikt

Harriet er 28 år gammel og har erfaring fra tidligere studier innen kulturarv og bevaringskunnskap. På fritiden liker hun å lese, strikke og være ute i naturen. Til høsten skal hun definitivt teste SoppSporer i skogen.

Brukerdokumentasjon

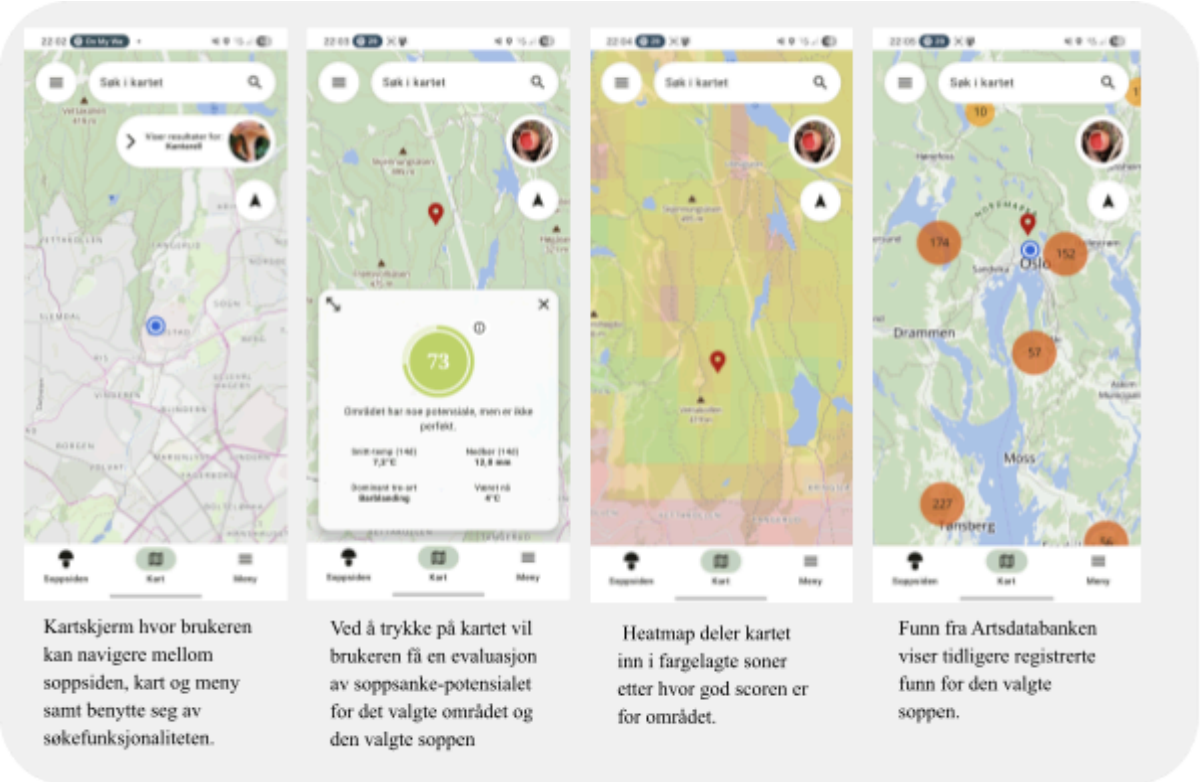
Hensikten med brukerdokumentasjonen er å forklare hva appen gjør, hvilke funksjoner den har, hvordan man bruker den og hvordan man kan aksessere appen. Brukerdokumentasjonen er skrevet for dem med og uten tekniske forkunnskaper, og vil derfor fokusere mer på de viktigste funksjonalitetene.

Målgruppe

Målgruppen for prosjektet er *personer som ønsker å plukke sopp i Norge*. Dette er en bred målgruppe som inkluderer både nybegynnere som vil lære mer om sopp, men også erfarne soppsankere som vil finne nye områder for å plukke sopp.

Funksjonalitet

SoppSporers hovedelement er en kartskjerm som brukeren kan navigere seg rundt i, enten ved å dra i kartet eller søke etter sted i søkebaren. Brukeren kan få en vurdering av et punkt ved å trykke på kartet, og blir da presentert for en score mellom 0-100 og muligheten til å utvide vinduet for en mer detaljert beskrivelse.



Kartskjerm hvor brukeren kan navigere mellom soppsiden, kart og meny samt benytte seg av søkefunksjonaliteten.

Ved å trykke på kartet vil brukeren få en evaluasjon av soppsanke-potensialet for det valgte området og den valgte soppen

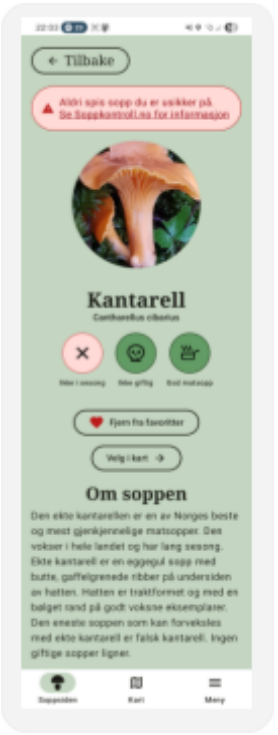
Heatmap deler kartet inn i fargelagte soner etter hvor god scoren er for området.

Funn fra Artsdatabanken viser tidligere registrerte funn for den valgte soppen.

Gjennom dropdown-menyen kan brukeren få tilgang til et heatmap som deler kartet inn i fargelagte soner etter hvor god scoren er for området. På den måten kan brukeren enkelt se grønne områder med mye potensial, uten å måtte klikke på kartet. Det er også mulig å se hvor det tidligere er registrert soppfunn fra Artsdatabanken.

For å navigere til andre skjermer i appen, benytter brukeren hovedmenyen nederst på skjermen. Soppsiden viser en oversikt over de 11 ulike sopptypene appen tilbyr, og herfra kan brukeren gå videre til hver sopp sin sopp-profil, der det tydelig presenteres hvorvidt soppen er i sesong, er giftig og om den er en god matsopp. Gjennom både soppsiden og hver sopp-profil kan brukeren også velge soppen i kartet for å se soppsankepotensialet for den soppen. Fra menysiden kan brukeren bytte mellom lyst og mørkt modus, ringe giftinformasjonen, velge algoritme for beregning, og lese mer om appen og hvordan den brukes.





Oppsummering av Funksjonalitet:

Funksjon	**Beskrivelse**
Kart	Navigerbart kart med funksjonalitet knyttet til Norge.
Søkebar	Muliggjør søk etter steder og adresser i Norge.
Sentraliser kart	Fokuserer skjermen til brukerens posisjon.
Evaluering av område	Ved trykk på et punkt i kartet vil brukeren få en evaluering av soppsankepotensialet for valgt sopp i form av en score fra 0-100.
Score card	Viser scoren fra evalueringen av området. Ved å utvide kortet kan man få mer detaljert informasjon om området, samt temperatur, nedbør, vind, bonitet, tresorter, samt andre faktorer som påvirker potensialet for sopp.
Dropdown-meny	Meny som inneholder heatmap, funn fra Artsdatabanken, skoglag, satellittmodus/kartmodus og eventuelle soppfavoritter.
Heatmap	Fargelegger kartet i nyanser fra grønt til rødt etter sankepotensialet i området. Bruker ML-algoritmen for beregning.

Funksjon	Beskrivelse
Vis funn fra Artdatabanken	Viser tidligere funn av valgt sopp fra Artsdatabanken som punkter/samlinger på kartet
Vis skoglag	Viser type skog som fargede felter på kartet.
Satellitt/kartmodus	Bytter mellom vanlig kartmodus og satellittmodus.
Favoritter	legges til under favorittseksjonen i soppsiden og tre utvalgte favoritter som hurtigvalg i kart, noe som muliggjør raskt bytte av sopp.
Soppsiden	Gir en generell oversikt over de ulike soppene du kan finne og favorittene dine. Gjør det mulig å velge hvilken sopp som vises i kartet. I tillegg kan brukeren navigere videre til sopp-profilene, samt til soppkontroll.no.
Sopp-profil	Her kan brukeren lese om soppen, i tillegg til å se om den er i sesong, er giftig og om den er en god matsopp. Man kan også legge til soppen som favoritt, eller velge å se soppen direkte i kartet.
Menyside	Inneholder både tilleggsfunksjonalitet og diverse informasjon.
Mørkemodus	Gjør det mulig å bytte mellom lyst og mørkt modus direkte i appen.
Ringe Giftinformasjonen	Gjør det mulig å ringe Giftinformasjonen via appen.
Velge beregningsalgoritme	Gjør det mulig å bytte mellom algoritmene ML, Forenklet og KI for beregning av score.

Plattformer og aksessering

Applikasjonen er utviklet i Android Studio og kan kun brukes av Android-enheter. Den er per nå ikke publisert for offentlig bruk, men prosjektet kan lastes ned via https://github.uio.no/IN2000-V26/team-16 med spesiell tillatelse. Følg stegene for nedlasting og kjøring der.





Produktdokumentasjon

Kvalitetsegenskaper ved appen

De fleste av brukerne involvert i utformingen og evalueringen av vår app er mennesker vi kjenner fra før, fra bekjente til nære relasjoner. Vi er bevisst på hvordan dette kan ha påvirket resultatet og ført til skjevhet i data, for eksempel ved at deltakerne har større sannsynlighet for å gi positive tilbakemeldinger eller unngå sterk kritikk. I tillegg kan utvalget ha blitt lite representativt for den faktiske målgruppen, fordi deltakerne gjerne deler bakgrunn, interesser, og/eller teknologisk kompetanse med oss i teamet.

Dette kan ha påvirket hvilke behov som ble identifisert, hvilken funksjonalitet som ble prioritert, samt evalueringen av brukervennlighet og design. Vi forsøkte å redusere denne biasen ved å bruke triangulering i datainnsamlings- og evalueringsmetoder, gjennom spørreundersøkelser, semi-strukturerte intervjuer og brukertester, samt å oppfordre deltakerne til å gi ærlige tilbakemeldinger.

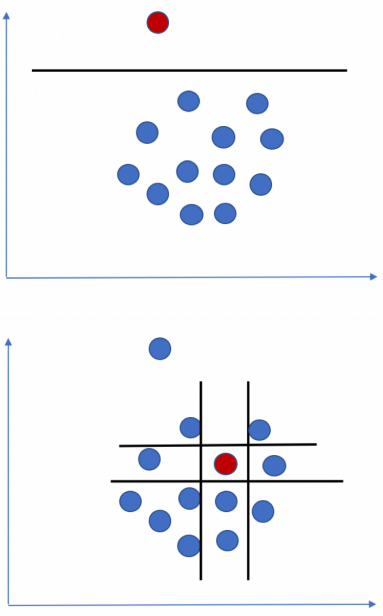
Funksjonalitet

Et kjerneelement i appens funksjonalitet ble bestemt på første møte, nemlig at appen skal tilby en kartfunksjon for beregning av sankepotensial for sopp. Videre ønsket vi å lage en app som støttet brukere i forbindelse med sopp-plukking, og appen tilbyr derfor en informasjonsside for sopp, hvor du kan få enkel informasjon om hver sopp, velge favoritter og velge hvilken sopp du vil bruke i kartet. Den tilbyr også en menyside med en switch-knapp for mørk/lys modus, nummeret til Giftinformasjonen, mulighet til å bytte beregningsalgoritme for sopp, guide til appen, personvernerklæring og informasjon om appen. Kartsiden tilbyr også heatmap, satellittfunksjon, visning av skoglag og tidligere soppfunn fra Artsdatabanken og hurtigvalg av sopp.

Funksjonalitetene ble utviklet og evaluert iterativt gjennom semi-strukturerte intervjuer, spørreundersøkelse og brukertester. De hjalp oss også å vurdere hvorvidt de faktisk svarte på brukernes behov, som svarer på ISO/EIC 25010-kriteriet *Functional suitability* (Sjøberg & Bergersen, 2026, lysark 18). Noen funksjonelle prioriteringer måtte teamet avgjøre selvstendig. For eksempel valgte vi å droppe muligheten til å lagre turer og egne sopp-funn (noe som var en ønsket funksjon hos flere brukere) fordi vi, grunnet mangel på tid og

ressurser, ønsket å prioritere det som fremsto som de viktigste funksjonene for brukerne (se vedlegg 6).

En sentral bit av appen er bruken av maskinlæringsmodeller for å predikere hvor bra et sted er for soppplukking. Det ble laget en maskinlæringsmodell for hver individuelle soppart for å kunne håndtere ulike krav. Disse modellene er lagret lokalt som ONNX filer og kjøres lokalt. Det ble brukt ubasert læring, eller mer konkret ble det brukt "Isolation Forest". Dette ble benyttet siden vi kun hadde positive funn å lære modellene opp på. Metoden går ut på å skille ut alle funn som skiller seg fra normalen ved å splitte opp datasettet på tilfeldige punkter. Jo flere ganger du må splitte opp dataene for å isolere et punkt, jo mer representativt er punktet. Punkter som er unormale og skiller seg ut vil derimot lettere kunne isoleres. Mer detaljer om koden kan leses om i vedlegg 3. (Mavuduru, 2021)



(Mavuduru, 2021)

Brukskvalitet

Vi ønsket å lage en app med et enkelt og intuitivt brukergrensesnitt som krevde lite opplæring, og slik støtte ISO/EIC 25010-kriteriet *Interaction Capability* (Sjøberg & Bergersen, 2026, lysark 18). I prototypingen forsøkte vi derfor å ta hensyn til etablerte designprinsipper for brukervennlighet:

- *Consistency*: For å skape et gjenkjennelig grensesnitt brukte vi kjente og standardiserte ikoner, og sørget for å bruke disse på en konsekvent måte i appen. Vi forsøkte også å opprettholde en konsekvent stil når det kom til farger og utforming.

  ☰ ⓘ → ⤢

- *Visibility*: Vi ønsket at sentrale funksjoner skulle være tydelige og enkle å oppdage. Det kunne vært en utfordring at den mest sentrale funksjonen er ganske usynlig, nemlig at du må trykke på kartet for å beregne score. Dette kunne vært løst ved å f.eks. legge inn en forklarende tekstboks på kartsiden, men vi ønsket å holde kartsiden så ryddig som mulig. Brukertestene skulle heldigvis vise at dette ikke var et problem, da brukerne intuitivt forsøkte å trykke på kartet som noe av det første de gjorde.

- *Feedback*: Appen gir brukeren tilbakemeldinger for å bekrefte at en handling er registrert. Dette løses gjennom loading-beskjeder og animasjoner, og at knapper gir feedback når du trykker på dem.
- *Affordance*: Knappene i appen er utformet slik at de fremstår klikkbare, blant annet gjennom avrundede bokser og standardiserte ikoner.
- *Constraints*: Vi forsøkte å begrense muligheten for feilbruk ved å holde grensesnittet enkelt og redusere antallet valg brukeren må ta samtidig. Funksjoner som ikke var nødvendige for kjerneopplevelsen ble derfor utelatt eller plassert i menyer for å unngå et overfylt grensesnitt. Dette bidro til å gjøre appen mer oversiktlig og enklere å bruke.
- *Mapping*: Vi ønsket at sammenhengen mellom brukerens handlinger og systemets respons skulle være naturlig og intuitiv. Et eksempel på dette er at brukeren trykker direkte på et område i kartet for å få beregnet sankepotensial akkurat der. Denne direkte koblingen mellom handling og resultat gjør funksjonaliteten lettere å forstå og lære.

Brukskvaliteten ble evaluert i brukertester. 3 enkle brukertester ble først gjennomført på Wireframes i Figma, og da appen var klar for det, gjennomførte vi én geriljatest og én planlagt brukertest på den. Det var dessverre ikke alle forbedringer vi rakk å gjøre fra wireframe-evalueringen til brukertestingen av appen, men siden vi da testet en funksjonell app istedenfor delvis interaktive skisser, kunne vi avdekke flere forbedringsområder som vi kunne jobbe med til ferdigstilling av prosjektet.

Pålitelighet

Vi ønsket å utvikle en stabil og robust app, i tråd med ISO/IEC 25010-kriteriet *Reliability* (Sjøberg & Bergersen, 2026, lysark 18). Dette innebar å forebygge feil, men også håndtere dem fortløpende når de oppsto. For å støtte oss i dette, brukte vi try-catch blokker og enkelte fallback-mekanismer i koden. Appen er utviklet for å støtte enkelte funksjoner også uten internettforbindelse, som ikke bare bidrar til økt robusthet, men gjør at brukeren kan benytte appen ute i felt hvor det kan være dårlig dekning, noe vi identifiserte som en vanlig brukssituasjon gjennom spørreundersøkelsen. Vær- og skogdata blir derimot utilgjengelig uten internett, som kan gi en misvisende score, og dermed reduseres funksjonaliteten noe. Vi utviklet også enhetstester for utvalgte deler av systemet for å kontrollere at det oppførte seg som forventet.

Pålitelighet ble evaluert gjennom manuell testing og enhetstester. Manuell testing ble brukt for å undersøke hvordan appen oppførte seg i ulike brukssituasjoner, inkludert håndtering av feil og ustabile nettverksforhold. Enhetstestene bidro til å verifisere at sentrale funksjoner fungerte korrekt isolert sett.

Universell utforming

Vi sørget også for at informasjon og funksjonalitet i appen skilte seg tydelig fra bakgrunnen, ved å bruke gode kontraster mellom tekst, ikoner og bakgrunnsfarger. Kontrastene ble kontrollert ved hjelp av verktøyet WebAIM Contrast Checker (WebAIM, 2026), og slik vurdert opp mot kravene i WCAG 2.1 nivå AA (i stor grad også AAA) om minimum kontrastforhold på 4,5:1 for vanlig tekst (UUtilsynet, 2026). Dette er viktig for universell utforming, og for å sikre at innholdet er lesbart og oppfattelig for brukere med nedsatt syn, men kan også bidra til å øke brukeropplevelsen for alle brukere. Det som kan være nødvendig for noen, kan være bra for alle (Herstad, 2026, lysark 7).

Med unntak av heatmap, er stedene i appen hvor fargene er sterkt knyttet til informasjonen supplert med tekst eller tall. Dette bidrar til at brukere med nedsatt fargesyn eller andre synsutfordringer kan forstå informasjonen. Dette gjelder ikonene på Soppsiden (i sesong / ikke i sesong, giftig / ikke giftig og god matsopp / ikke god matsopp), score-verdien i kartet, og historiske funn, samt advarsels-boksene. Bilder og ikoner har tilhørende alternativ tekst i koden for å øke tilgjengeligheten. Vi har ikke laget en tilgjengelighetserklæring, men dette er noe vi kommer til å utvide dersom appen blir tilgjengelig offentlig.

Utvalget av brukere involvert i prosjektet har også en underrepresentasjon av veldig unge og eldre brukere, samt brukere med ulike funksjonsnedsettelser. Vi har derfor ikke tatt tilstrekkelig hensyn til den brede målgruppen vi har, og vil også rette tydelig fokus mot dette i eventuell videreutvikling av appen.

Sikkerhet

Vi har gjort enkelte sikkerhetstiltak for å gjøre appen vår trygg å bruke, både når det gjelder personvern og risiko knyttet til soppsanking. Appen vår samler ikke inn personopplysninger, da brukerens posisjon og favorittsopper behandles og lagres lokalt på brukerens enhet. Dette reduserer risikoen for misbruk av brukerdata og bidrar til å ivareta personvern, som støtter ISO/IEC 25010-kriteriet *Security* (Sjøberg & Bergersen, 2026, lysark 18).



Vi har også lagt stor vekt på sikkerhetsmessige hensyn knyttet til selve soppsankingen. Siden feil identifisering av sopp kan medføre alvorlig helserisiko, presiserer appen tydelig at den ikke skal brukes som et verktøy for identifisering av sopp. Appen er utviklet som et hjelpemiddel for å finne områder med sannsynlig sankepotensial og tilbyr enkel informasjon om sopp, men kan ikke brukes for å avgjøre om en sopp er trygg å spise.

For å gjøre brukeren oppmerksom på dette har vi lagt inn tydelige advarsler i appen om at soppsanking innebærer risiko, og at brukeren selv har ansvar for å kontrollere sopp hos en kvalifisert soppkontrollør. Vi har derfor fått tillatelse av Norges sopp- og nyttevekstforbund til å videreføre brukeren til deres tjeneste Soppkontroll.no inne i vår app. I tillegg har vi gjort kontaktinformasjon til Giftinformasjonen lett tilgjengelig i menyen, slik at brukeren raskt kan få hjelp ved mistanke om forgiftning.

Bruk av API-er

MET-Frost: MET-Frost brukes for å hente historisk værdata, spesifikt for å sjekke minimumstemperaturer, maks temp og regn de siste 14 dagene. Dette brukes i både "kill switch"-logikken for å avgjøre om frost har tatt livet av soppen i et område, og for selve algoritmen. Frost-API-ets innebygde funksjon for å finne "nærmeste stasjon" basert på koordinater var ekstremt treg (responstid på 8–10 sekunder). Dette førte til en dårlig brukeropplevelse i kartvisningen. For å løse denne utfordringen implementerte vi en egen StationLocator. Ved å lagre en oversikt over alle norske værstasjoner i en lokal ressursfil (weatherstations.csv), kan appen beregne den nærmeste stasjonen lokalt (ved hjelp av Haversine-formelen) før API-et kalles med en spesifikk stasjons-ID. Dette reduserte responstiden til under 1 sekund.

MET-LocationForecast:
MET-LocationForecast brukes for å hente sanntidsdata og værvarsel for brukerens posisjon. Appen benytter 'complete'-endepunktet. Dette ble valgt fordi det gir tilgang til mer avanserte parametere som tåke og duggpunkt. Dette er viktige indikatorer for fuktighetsforhold i skogbunnen som påvirker soppvekst. MET krever en unik User-Agent i alle forespørsler for å

overholde deres retningslinjer. Dette ble løst ved å legge til en egen header (IN2000 Team 16) i HTTP-klienten for å sikre stabil tilgang.

GBIF: (Global Biodiversity Information Facility). Dette APIet var tiltenkt å brukes for å hente informasjon om forskjellige sopper. Vi fant fort ut at informasjonen man fikk ut fra søk var veldig teknisk, det var mye unyttig informasjon som ikke var relevant for vårt formål. Vi valgte derfor å bruke dette APIet for å hente ut historiske soppfunn gjennom GBIF, som er en global infrastruktur som samler biodiversitetsdata fra hele verden. Mye av de norske observasjonsdataene i GBIF kommer opprinnelig fra norske kilder som Artsdatabanken.

I starten var det utfordrende å forstå søkestrukturen, ettersom man kan søke på flere taksonomiske nivåer (familie, art, slekt osv.). For oss uten biologisk bakgrunn krevde dette litt innsats å sette seg inn i, men etter at vi fikk oversikt, var APIet relativt enkelt å bruke.

Vi merket oss etter hvert at nettverkskallene noen ganger feilet. Siden vi hadde valgt at appen skulle inneholde 11 sopper, sjekket vi hvor mye plass det ville ta å laste ned alle funnene for tiltenkte sopper i Norge. Vi fant ut at det bare tok rundt 5 MB, derfor valgte vi å gå bort fra API kallene og heller hente det fra en intern fil. Vi laget et script som lastet ned alle funnene på forhånd. Vi lagret denne dataen i en JSON fil, der hver sopptype ble identifisert med et heltall (Int) for å spare plass. Denne løsningen gjorde appen betydelig raskere og sikret at vi alltid fikk treff på historiske funn, uavhengig av nettverkstilkobling.

NIBIO SAT-SKOG: NIBIO SAT-SKOG gir detaljert informasjon om skogens sammensetning, inkludert treslag, skogens alder og fordeling av gran, furu og lauvskog. Siden mange sopparter er knyttet til bestemte trær, er dette kritisk for algoritmen. En utfordring var at dette er en WMS-tjeneste (Web Map Service) som primært er laget for å vise kartbilder, ikke for å returnere rådata til applikasjoner. Vi løste dette ved å bruke GetFeatureInfo. Ved å sende inn en bitteliten boks (BBOX) rundt brukerens koordinater og be om info i GML-format, kan appen trekke ut tekst- og tallverdier fra kartet i stedet for å bare vise et bilde.

NIBIO AR5: NIBIO AR5 brukes for å hente nøyaktig arealtype (f.eks. om et punkt faktisk er skog, beite, eller bebygd område). I likhet med SAT-SKOG er dette en karttjeneste som krever kompleks parsing for å få ut data. For å løse denne utfordringen brukte vi samme teknikk som med SAT-SKOG, der vi parser GML-responsen med regulære uttrykk (Regex) for å hente ut verdien i *artype_beskrivelse*. Dette gjorde det mulig å filtrere ut uegnede områder (som bykjerner eller vann) fra heatmap-beregningen.

Kartverkets Adresse REST-API: Dette APIet kan brukes for oppslag av offisielle adresser i matrikkelen (Statens kartverk, u.å.a). Adresser kan søkes etter tekstlig eller etter nærhet til punkt. Vi har brukt APIet for at brukeren skal kunne søke etter adresser i appens søkebar, ved å skrive inn adressens navn. Dette APIet var enkelt å bruke, og vi støtte ikke på noen utfordringer.

Kartverkets Stedsnavn REST-API:
Dette APIet ligner det ovennevnte, men kan brukes for oppslag av stedsnavn i Norge (Statens kartverk, u.å.b). Også her kan steder søkes etter tekstlig eller etter nærhet til punkt. Vi brukte dette APIet for at brukeren ikke bare skulle kunne søke etter adresse, men også steder. I sopplukkingssammenheng er brukeren ofte mer interessert i skogs- og naturområder enn nøyaktige adresser, og vi bestemte oss for at mulighet for søk etter både adresser og steder ville være mest brukervennlig. Ved bruk av dette APIet var det litt utfordrende å forstå hvilket resultat de ulike feltene ga, så her måtte vi prøve å feile litt for å finne ut hvilket variabelnavn som inneholdt den informasjonen vi var ute etter.

Prosessdokumentasjon

Utviklingsmetode og bruk av smidige praksiser

I vårt prosjekt brukte vi utviklingsmetoden Scrum, med tidsbestemte sprinter og board/tavle for å visualisere oppgaver.

Roller

Vi ønsket å ha en flat struktur i teamet, og anså derfor hele teamet som produkteier. Vi rullerte hver uke på hvem som skulle fungere som en slags Scrum Master, som hadde ansvar for å booke grupperom og sørge for at vi kom oss gjennom agendaen for møtet.

Sprinter

Selv om det er vanlig at sprinter varer i minst 2 uker, valgte vi å ha sprinter på 1 uke på grunn av tidsbegrensningen. Vi startet med fast møtetid på torsdager, men endret møtetid til hver onsdag og fredag. Da møtte vi opp fysisk på instituttet for å støtte opp om smidig prinsipp nr. 6 (Lindstrøm, 2026, lysark 29). Hvert møte startet med *daily stand-up*, hvor vi gikk gjennom hva vi hadde gjort siden sist, hva vi skulle jobbe med videre, om vi hadde møtt på noen hindringer, etc. Siden vi merket at det gikk lenge mellom fredags- og onsdagsmøtet, så vi behovet for å innføre en "digital stand-up" på Discord, hvor alle i løpet av mandag eller tirsdag oppdaterte teamet på sin progresjon.

Hvert fredagsmøte markerte slutten av én sprint og starten av neste, så mye av tiden ble da brukt på å diskutere forrige sprint i uformelle sprint reviews, og til å planlegge neste sprint og gjøre backlog refinements i sprint planning. Vi holdt også to sprintretrospektiver i løpet av prosessen hvor vi satte av mer tid til å diskutere hva vi gjorde bra, hva vi gjorde mindre bra og viktigst av alt hva vi skulle ta med oss videre eller forbedre i prosjektet.

Verktøy

Github: Github ble brukt for versjonshåndtering. Vi benyttet oss av en forenklet Gitflow branching-strategi, der *working* fungerte som integrasjons-branch og *main* fungerte som en stabil branch som kun fikk fungerende kode (Almås, 2026, lysark 20). Arbeidet i working-branchen var inspirert av en trunk-based strategi med kortlevende branches som vi opprettet og slettet hyppig (Almås, 2026, lysark 22).

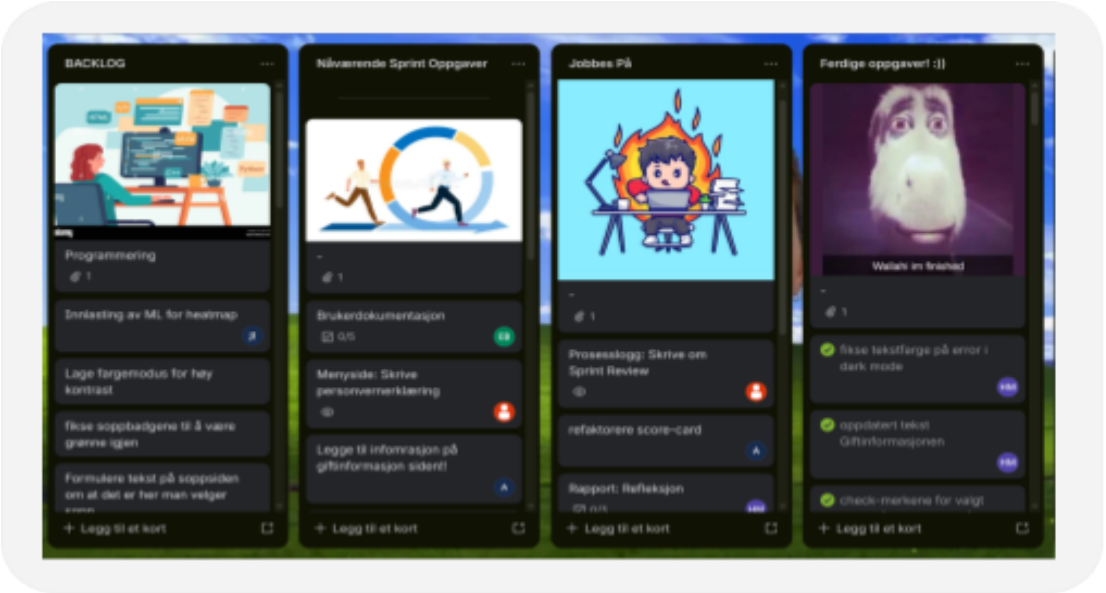


Discord: Som kommunikasjonsplattform i teamet brukte vi Discord, hvor vi opprettet tematiske kanaler for ulike formål.

Google Drive: For håndtering og deling av filer og dokumenter som møtenotater, prosesslogg og datainnsamling brukte vi Google Drive.

Trello: For visualisering og fordeling av oppgaver valgte vi å bruke en blanding av Scrum- og KanBan-board i Trello. Vi prioriterte og fordelte oppgaver hver fredag, men la opp til at vi kunne velge nye oppgaver fra backlogen dersom man ble ferdig med sine oppgaver før sprinten var ferdig, et prinsipp lånt fra KanBan. Samtidig la vi ikke begrensninger for hvor mange oppgaver som kunne jobbes på om gangen, i mer kjent Scrum-stil (Miro, 2026).

Kjerne-listene var backlog (alle oppgaver vi hadde identifisert så langt), nåværende sprintoppgaver (oppgaver prioritert for gjeldende sprint), jobbes på (påbegynte oppgaver) og ferdige oppgaver, satt opp slik:



Da vi innså at vi hadde flere ønskede funksjoner enn vi sannsynligvis kom til å få implementert på en god måte, lagde vi en egen liste kalt “Ønskede funksjoner”. Hit flyttet vi mindre kritiske funksjoner vi gjerne ville utvide med om vi fikk tid. Vi så det også som hensiktsmessig å opprette egne lister for bugs og fiksede bugs da disse hoppet seg opp.

Oversikt over prosjektets gang

Det er mulig å dele vårt prosjekt i 4 iterasjoner, hvor hver iterasjon inneholder aktiviteter, innsikt og konsekvens for løsningen.

Iterasjon 1: Innsiktsarbeid og konseptutvikling

Iterasjon 1 handlet om å finne ut *hva* vi skulle lage. På første møte ble det bestemt at vi skulle lage en app for sopp-plukking. Én på gruppa kom med forslaget og hadde allerede funnet relevante API-er som kunne brukes til å si noe om vekstforholdene til ulike sopper. Vi definerte et sett med krav, og begynte å sette opp et kart og hente data fra relevante API-er. For å sørge for at dette ikke ble et rent *genius design*, ønsket vi videre å tilpasse denne løsningen til brukernes og målgruppens ønsker og behov. Vi hadde derfor en runde med innsiktsarbeid i form av semi-strukturerte intervjuer for å kartlegge eksisterende vaner, preferanser og holdninger knyttet til sopplukking, samt bruk av både teknologiske og ikke-teknologiske hjelpemidler i den sammenheng.

Det ble gjennomført 5 intervjuer som ble transkribert og analysert i tematisk analyse. Funn fra analysen ble samlet i et affinitydiagram (se vedlegg 4), som dannet grunnlaget for ytterligere kravspesifikasjon og en design-workshop innad i teamet. Ved hjelp av workshopen Crazy 8 fikk teamet idémyldret løsninger, laget enkle skisser basert på de nye funnene og kravene, og dannet utgangspunktet for Kartsiden, Soppsiden og Min side.

For enda mer smidig og brukerstyrt utvikling kunne vi evaluert disse tidlige prototypene med brukerne. Istedenfor ble vår tenkte funksjonalitet evaluert i den senere spørreundersøkelsen, for eksempel ved å spørre hvor nyttige ulike funksjoner ville være for dem (se vedlegg 6). Dette ga oss derfor et kvantitativt supplement til intervjuene, og et grunnlag for å prioritere videre utvikling.

Etter første iterasjon satt vi igjen med innsikt i målgruppens behov og utfordringer, grunnleggende app-kode, tidlige konsepter og skisser, en foreløpig kravspesifikasjon, og en overordnet idé om appens kjernefunksjonalitet.

Iterasjon 2: Validering og prototyping

I den andre iterasjonen var fokuset på å konkretisere og validere valg knyttet til funksjonalitet, samt å utforme og evaluere designforslag. Vi gjennomførte en

spørreundersøkelse, hvor rekruttering ble gjort ved å dele undersøkelsen med bekjente og legge den ut på sosiale medier. Målet med spørreskjemaet var å lære mer om brukernes behov og ønsker til appen, vurdere brukssituasjoner og evaluere hvorvidt vår hittil tenkte funksjonalitet ville imøtekomme deres behov og ønsker. Parallelt jobbet utviklere med å få på plass viktige funksjonaliteter i koden, som heatmap, AI-algoritme, bedre log-funksjoner og allerede bestemte ui-komponenter.

Spørreundersøkelsen hjalp oss spesielt med å prioritere og justere funksjonaliteten vi burde gå videre med, og hvilke funksjonaliteter vi kunne nedprioritere/fjerne helt. Vi valgte f.eks. å endre Min side til en menyside, da dette var en mindre kritisk funksjon for brukerne, og vi ikke hadde tid til å implementere alt. Med utgangspunkt i den innsamlede dataen og våre egne vurderinger, ble det laget wireframes i Figma, som ble evaluert i 3 enkle brukertester.

Det ble også gjennomført et intervju med en soppekspert, som kom med tips og hjalp oss med vektleggingen i algoritmen, samt prioritering av hvilke sopp som burde være med i appen.

Etter andre iterasjon satt vi igjen med en mer tilspisset løsning, samt en bredere forståelse av brukerne.

Iterasjon 3: Videreutvikling og brukervennlighet

Iterasjon 3 handlet hovedsakelig om å forbedre brukervennligheten og utvide appen. Justeringer og valg ble gjort med utgangspunkt i funn fra brukertestene, med etablerte designprinsipper for interaksjonsdesign i bakhodet. For eksempel var det én av deltakerne i brukertesten som påpekte at det var lurt å ha god konsekventhet i navigasjonsbaren, som gjorde at vi valgte det ene designforslaget over det andre.

Deretter ble selve appen for første gang brukertestet, først i én geriljatest og deretter i én avtalt brukertest. Resultatene fra disse førte til flere justeringer i både design og funksjonalitet for å gjøre appen mer intuitiv og brukervennlig. Vi jobbet da særlig med å forbedre visualiseringen av valgt sopp og funksjonaliteten rundt hvordan den velges, samt bedre forklaring av score-systemet.

Iterasjon 4: Ferdigstilling og stabilisering

Siste iterasjon handlet hovedsakelig om å ferdigstille appen. Dette innebar justeringer av design basert på brukertestene, men også mye refaktorering og debugging for å sikre appens

robusthet og pålitelighet og redusere teknisk gjeld. I tillegg ble det gjennomført et intervju med enda en soppekspert for å kvalitetssikre enkelte faglige vurderinger og diskutere videre forbedringsmuligheter for appen for eventuell videreutvikling etter prosjektets slutt.

Endringer i kravspesifikasjonen

Kravspesifikasjonene ved prosjektets start og slutt kan leses i vedlegg 2. Vi gjorde noen mindre endringer i de funksjonelle kravene vi etablerte ved prosjektstart, hvor de gjeldende kan ses i tabellen nedenfor. FK-5 var et krav vi i utgangspunktet stilte i kravspesifikasjonen, da vi ønsket å inkludere terrenghøyde som en variabel i beregning av score for soppsannsynlighet. Grunnet prosjektets tidsbegrensning valgte vi imidlertid å fjerne dette kravet.

Når det gjelder FK-10 bestemte vi oss ganske tidlig i prosjektet for å snevre inn appens omfang til å kun omhandle sopp, ikke bær, og vi modifiserte kravet deretter. Vi ønsket å ha en tydelig retning for appen vår, spisse inn produktet og ikke gape over for mye. Å utvide appen til å inneholde bærprofiler er noe vi eventuelt kan utvide med i fremtiden.

FK-13 var et krav vi måtte justere når vi så klarere for oss hvordan appen skulle fungere og se ut for brukeren. Ved prosjektstart så vi for oss at dersom brukeren ikke filtrerte på ønsket sopp, ville man kunne lete etter soppmuligheter generelt. Da dette var tanken, ønsket vi krav om at brukeren skulle kunne filtrere ut valgte sopper. Etter å ha kommet i gang med prosjektet ønsket vi å gi hver sopp sin egen profil, med sin egen tilpassede beregningsalgoritme. Nå ga det ikke lenger mening å filtrere på sopp, men å se resultater for én sopp av gangen. Dersom vi fikk tid ønsket vi å utvide appen til å kunne se muligheter for vekst for flere sopptyper om gangen, men dette fikk vi ikke tid til grunnet prosjektets korte tidsramme. Vi endret derfor kravet slik tabellen viser.

Krav vi endret fra prosjektstart til -slutt:

FK-5	Appen skal bruke GPS-koordinater til å hente terrenghøyde (fra Kartverket)
FK-10	Bruker skal kunne velge sopp (og ev. bær) fra flere profiler
Justert FK-10	Bruker skal kunne velge sopp fra flere profiler
FK-13	Bruker skal kunne filtrere på sopptyper
Justert FK-13	Bruker skal kunne bytte hvilken sopp som det vises resultater for

Utover disse endret vi ingen av kravene i kravspesifikasjonen, men en rekke krav tilkom som resultat av spørreundersøkelser, intervjuer, brukertesting og geriljatesting. FK-14 til FK-25 og IFK-14 til IFK-16 ble spesifisert ut ifra funn fra disse innsiktsarbeidene, samt realiseringer vi i teamet hadde underveis.

Testing av løsningen underveis

Brukertestene vi holdt var enkle og relativt lavterskel. Siden vi ønsket innblikk i hvordan brukerne naturlig møter appen og forstår funksjonaliteten og grensesnittet, ønsket vi å legge så lite føringer som mulig og ga derfor ingen spesifikke oppgaver i starten, men lot dem heller utforske på egen hånd. Underveis observerte vi hvordan de navigerte, hvilke funksjoner de forstod intuitivt, og hvor de eventuelt møtte utfordringer eller usikkerhet. Vi stilte også oppfølgingsspørsmål underveis for å få innsikt i tankene deres og hvorfor de tok bestemte valg.

Test av prototyper/wireframes (3 personer)

Mål: Å evaluere designvalg og funksjonalitet

Verktøy: Figma og iPhone

Gjennomføring: Deltakerne fikk "bruke" prototypene (se vedlegg 5) selv i Figma, selv om de ikke var 100% interaktive. De ble bedt om å forklare det de oppfattet på skjermen, hvordan de tolket ulike elementer, samt hva de forventet skulle skje når de interagerte med ulike funksjoner. Vi oppfordret dem som sagt til å tenke høyt underveis, og under og etter testen stilte vi spørsmål om hvilke løsninger de foretrakk, hva som virket intuitivt, og hvilke designvalg som eventuelt skapte forvirring eller usikkerhet.

Geriljatest (1 person)

Mål: Å evaluere funksjonalitet og brukervennlighet. Vi var særlig interessert i om hvordan de skjønte scoren, og om de skjønte hvordan de skulle velge sopp til beregning.

Verktøy: Android-mobil, emulator i Android Studio, notat-app på mobil

Gjennomføring: Deltakeren fikk bruke appen fritt mens de tenkte høyt og forklarte hva de gjorde og hvorfor. Underveis stilte vi oppfølgingsspørsmål for å avdekke hva de forstod, hva som eventuelt var uklart, og hvilke forventninger de hadde til funksjonaliteten.



Brukertest av app (1 person)

Mål: Å evaluere funksjonalitet og brukervennlighet. Vi var særlig interessert i om de forstod scoren, og om de skjønte hvordan de skulle velge sopp til beregning.

Verktøy: Android-nettbrett, emulator i Android Studio, notater i Google Docs.

Gjennomføring: Deltakeren fikk bruke appen på egen hånd mens de tenkte høyt underveis. Vi observerte hvordan han løste oppgaver naturlig gjennom utforskning av appen, og fokuserte særlig på hvor intuitiv navigasjonen og funksjonaliteten opplevdes. Underveis stilte vi spørsmål om hvordan de oppfattet ulike funksjoner, om de forstod scoringssystemet, og om det var elementer som skapte usikkerhet eller frustrasjon.

Automatisk enhetstest

Mål: Å forsikre oss om at appens funksjonalitet fungerer som tenkt, og å kunne bruke disse testene underveis for å passe på at endring i koden ikke forårsaker uforutsett oppførsel i appen.

Verktøy: Android Studio

Gjennomføring: Testene ble kjørt ved behov og ved større endringer i relevante deler av koden. Det var nødvendig å endre testene underveis slik at forventede parametere og returverdier i testene samsvarte med utvidelse og endringer i koden.





Refleksjon

Tverrfaglig arbeid

Teamet vårt bestod av tre utviklere og tre designere, men hvor to av designerne i praksis har fungert som utviklere. Vi var i stor grad et kryssfunksjonelt team heller enn et komponent-team fordi de fleste teammedlemmene ikke har vært låst til faste områder av utviklingen, men hatt muligheten til å bidra der det trengs (Lindsjørn, 2026, lysark 27). Det har vært en stor fordel for oss å være et tverrfaglig team, og vi har virkelig dratt nytte av teammedlemmenes ulike erfaringer og innsikter. Å ha tre designere på teamet har gitt oss muligheten til å ha mye fokus på brukervennlighet og brukerrettet design i utviklingen, og utviklerne har også vært inkludert i denne tankeprosessen. Samtidig har utviklerne formidlet hva som har vært mulig å få til på gjeldende stadium i utviklingen og gitt designprosessen rammer.

De utviklerne som i begynnelsen jobbet utelukkende i back-end har tatt på seg flere front-end-oppgaver mot prosjektets slutt for å hjelpe til med å ferdigstille produktet. Teammedlemmene har hatt sine ansvarsområder, men det har også vært åpent for at noen kan jobbe videre på andres kode. Å legge til rette for dette har bidratt til at vi har ønsket å skrive lesbar kode med beskrivende variabel- og funksjonsnavn og forklarende kommentarer. Det har vært nyttig å gjøre koden så tilgjengelig som mulig for alle teammedlemmene da det ble enklere å få hjelp til å løse problemer, og å fordele arbeidskraft der den trengtes mest.

Én av designerne våre har stort sett arbeidet med designoppgaver som for eksempel intervjuer, brukertesting og wire-framing. De andre på teamet har vært involvert og oppdatert på disse oppgavene, men ikke bidratt til dem i større grad. For det designansvarlige teammedlemmet har vi i større grad fungert som et komponent-team, fordi denne designeren har fokusert på én del av utviklingsprosessen (Lindsjørn, 2026, lysark 26). Selv om det har gjort det mer utfordrende for designeren å sette seg inn i koden, har det vært en stor fordel for utviklingen at vi har hatt en designansvarlig på teamet. Det har gitt oss en visuell profil å arbeide ut ifra og felles forståelse i teamet om hvilke funksjoner vi trenger og hvordan de skal presenteres for brukeren.

I teamet vårt fordelte vi ikke tydelige ansvarsområder fra start, men lot disse vokse frem organisk ettersom prosjektet utviklet seg. Vi opplevde at dette gjorde oss mer åpne for å



fordele oppgaver på en effektiv måte utover i prosjektet. Jo lenger i arbeidet vi kom, jo tydeligere ble det for oss hvilke typer oppgaver som hang sammen. En person som hadde satt seg inn i et område og var god på det, kunne ta på seg flere oppgaver av samme type. Å fordele oppgaver på denne måten førte også med seg noen begrensninger, som f.eks. at en person fikk mer eierskap til en del av koden enn de andre, men samtidig er det en naturlig del av teamarbeid og ansvarsfordeling. Med åpen kommunikasjon og et ønske om å bistå hverandre, førte ikke dette til problemer for oss.

Smidig utviklingsmetode

Som beskrevet i prosessdokumentasjonen fulgte teamet Scrum. Vi var opptatt av å gjennomføre de smidige prosessene som var meningsfulle for oss, og teamaktivitetene ga oss mye inspirasjon til å utføre ulike typer smidige metodikker. I tillegg hjalp disse aktivitetene oss med å jobbe smidig jevnt over prosjektperioden. Å jobbe smidig var nytt for alle i teamet, og vi lærte mer og mer om hvordan smidig arbeidsmetode fungerer underveis i prosjektet. Teamet har vært flinke til å planlegge sprinter og har stort sett fått til å gjennomføre alle oppsatte oppgaver innenfor den satte tidsperioden. Vi startet også med rullerende rolle som Scrum master, men når alle teammedlemmene hadde hatt rollen én gang hver, glemte vi bort praksisen litt. Antakelig skjedde dette fordi vi opplevde at Scrum master i praksis ikke hadde noen funksjon i teamet. Ettersom alle i teamet naturlig bidro til god samtale- og arbeidsflyt, ble ikke en rolle som fasilitator nødvendig for oss (Lindsjørn, 2026, lysark 49).

Noe vi gjerne skulle ha vært flinkere til var å holde konkrete retrospektiver etter hver sprint, i tillegg til sprint review og sprint planning. De to gangene vi gjennomførte denne metoden ordentlig, kom det frem forbedringspotensialer som var nyttige å dele og diskutere. Hvis vi hadde vært flinkere til å gjennomføre et slikt møte som avslutning av hver sprint, ville vi nok ha unngått å støte på gjentagende problemer, slik vi av og til gjorde. Når det er sagt, var vi jevnt over åpne med hverandre om hva vi syntes var bra og hva som kunne fungere bedre. Vi nedprioriterte retrospektive møter da møtetiden vår ofte var litt knapp, og vi trengte tiden til andre ting. Et av prinsippene i smidig utvikling er at individer og interaksjoner skal tas hensyn til over prosesser og verktøy, og Ian Sommerville mener derfor at et team bør kunne tilpasse og modifisere smidige prosesser til sine omstendigheter (Sommerville, 2020, s. 31). Vi mener derfor at vi har fått til å jobbe smidig selv om vi ikke for hver eneste sprint har fulgt alle prosessene til punkt og prikke. Teamet vårt har tilpasset de smidige metodene etter våre



behov og brukt de som verktøy for å optimalisere teamarbeidet, heller enn å inkludere dem der det ikke har vært hensiktsmessig.

Effekten av å jobbe smidig har vært svært positivt for teamarbeidet og prosjektet. Den smidige utviklingsmetoden har gitt oss uvurderlige verktøy for å følge med på prosessen og hvor de ulike teammedlemmene er i den. Vi har dratt stor nytte av å bruke et Scrum-board og backlog, da vi har kunnet ha oversikt over våre egne og andres oppgaver, og tatt på oss flere oppgaver etter behov. Å holde stand-up ofte har også hjulpet oss mye i å være oppdatert på prosjektarbeidet og å komme med ros og tilbakemeldinger til hverandre. Det har vært motiverende for teamet å se at vi hele tiden beveger oss fremover. Retrospektivene har gitt oss mulighet til å tenke kritisk over prosessen vår og gjort at teammedlemmene er på samme bølgelengde. Vi tror at den smidige utviklingsmetoden har gjort det lettere å gjennomføre prosjektet og gitt oss mulighet til å jobbe mer effektivt og produktivt enn om vi ikke hadde jobbet smidig.

Utfordringer

Kartvisning

I løpet av utviklingsprosessen støtte vi på flere utfordringer. Som regel dreide disse seg om arbeidsteknikk og kommunikasjon, men vi har hatt et større problem med koden, som krevde flere teammedlemmer for å løse. Problemet omhandlet hvordan AndroidView viser kartet fra MapLibre i appen. Vi opplevde problemer med et Surface-mode av kartet, da det resulterte i en krasj i mørkemodus ved bytte mellom vår app og andre apper. Derfor gikk vi over til å bruke et Texture mode, som er en mer robust visning. Denne modusen ga oss imidlertid et nytt problem, hvor kartet av og til ikke strakk seg helt ned til navigasjonsbaren i bunnen av skjermen. Når dette skjedde, viste skjermen et smalt tomrom mellom navigasjonsbaren og kartet. Løsningen ble å la kartet strekke seg helt ned til bunnen av skjermen, bak navigasjonsbaren.

Møtelengde

I første halvdel av prosjektet hadde vi kortere møter, og vi opplevde at tiden ofte ble knapp og at møteagendaene ble så lange at vi ikke fikk diskutert gjennom ting så grundig som vi gjerne skulle ha gjort. Etter noen uker bestemte vi oss for å utvide møtetiden, men da hadde vi en utfordring med at møtene ble for lange. Dette løste vi ved å legge inn et fast



pausetidspunkt i midten av møtet, for å spise og hente ny energi. På denne måten fikk vi tid til å gå gjennom alt vi trengte på hvert møte, uten at vi ble altfor slitne.

Oppdatering av Scrum-board

Fra prosjektstart var vi flinke til å skrive ned oppgaver i backlog og flytte oppgaver for sprinten inn på Scrum-boardet vårt, men vi innså etter en stund at vi måtte bruke boardet enda mer aktivt. Mot slutten av prosjektet var vi mange som gjorde små oppgaver i ulike deler av koden, og det ble nødvendig at man var tydelig på hvilke oppgaver som var under arbeid og hvilke som var ferdigstilte. Ved å oppdatere dette i sanntid, kunne resten av teammedlemmene vite med sikkerhet at en oppgave som ikke var under arbeid var en oppgave man kunne ta på seg. Dette førte til mer effektiv arbeidsflyt og kommunikasjon.

Merge-konflikt

I begynnelsen av prosjektet opplevde vi oftere merge-konflikter med Git, som vi brukte litt tid på å løse i fellesskap. Etter hvert som vi fikk mer erfaring med versjonshåndtering, forstod vi at vi burde merge oftere for å unngå konflikter. Vi fant ut at det var en fordel å merge til working-branchen selv om man ikke var helt ferdig med oppgaven man holdt på med i en oppgave-branch, slik at ikke working-branchen beveget seg for langt i forhold til oppgave-branchen. For å unngå konflikter begynte vi også å merge working-branchen inn i de ulike oppgave-branchene for å holde dem oppdatert. Disse grepene bidro til at vi utover i prosjektet fikk færre merge-konflikter.



Avslutning

Prosjektarbeidet i IN2000 har gitt oss mange ulike og verdifulle erfaringer og ferdigheter som vi tar med oss videre. Under følger det viktigste hver av oss kommer til å ta med oss videre fra prosjektarbeidet i IN2000.

Adam: Jeg har erfart mye gjennom IN2000 prosjektet, fra å jobbe fullstack i kodebasen, til å avtale samarbeid med andre tjenester og møter med eksperter, men spesielt har det vært ekstremt verdifullt å erfare hvordan det er å arbeide på et større prosjekt i lag som utvikler. Det har vist meg hvor utrolig viktig et godt arbeidsmiljø og en åpenhetskultur for ideer og kritikk er, som grunnlag for et godt produkt.

Esma: Gjennom prosjektet har jeg lært viktigheten med en streng møteagenda og god kommunikasjon. Prosjektarbeidet har vært spennende og en fantastisk mulighet til å lære hvordan man arbeider som utvikler.

Jostein: IN2000 har vært et spennende prosjekt hvor vi har fått muligheten til å fordype oss i et prosjekt. Jeg tror at noe av det viktigste jeg vil ta med meg videre er hvor lurt og stor gevinst vi har fått av å holde prosjektet ryddig og oversiktlig fra starten og ikke utsette dette til avslutningen.

Mikael: Gjennom prosjektet har jeg fått mange nye erfaringer. Hvordan man jobber sammen rundt en kodebase og viktigheten av god kommunikasjon innad i et team er noe jeg kommer til å ta med meg videre.

Harriet: Det viktigste jeg tar med meg fra prosjektarbeidet er erfaring med smidig arbeid i team. Å bidra til et velfungerende samarbeid med god kommunikasjon og arbeidsflyt vil være veldig viktig i arbeidslivet som utvikler, og jeg er veldig glad for å ha vært med på et så godt samarbeid som vi har hatt i teamet vårt i dette emnet.

Heidi: I dette prosjektet har jeg fått mer erfaring med tverrfaglig samarbeid i et sammensatt team, og fått god trening i planlegging og gjennomføring av viktige oppgaver i interaksjonsdesign. I tillegg har jeg fått erfare viktigheten av å jobbe smidig, med god planlegging, struktur og kommunikasjon i teamet.



Felles tips til neste års studenter:

Sett av tid i starten til å konkretisere hva alle vil få ut av prosjektet (gjerne skriftlig), hvilken måloppnåelse man sikter til og hvor mye man har mulighet til å legge inn i prosjektet. Si ifra så fort som mulig og vær ærlig om uenigheter, men hold disputter saklig.



Referanseliste

Almås, S. (2026). *Git, API og dataformater.* [Lysarkpresentasjon]. https://www.uio.no/studier/emner/matnat/ifi/IN2000/v26/forelesninger/in2000.2026.01.30_git-api-dataformater.pdf

Herstad, J. (2026). *Tilgjengelighet og Universell Utforming* [Lysarkpresentasjon]. https://www.uio.no/studier/emner/matnat/ifi/IN2000/v26/forelesninger/in2000uu2026.pdf

Lindsjørn, A. (2026) *Teamarbeid i software engineering* [Lysarkpresentasjon]. https://www.uio.no/studier/emner/matnat/ifi/IN2000/v26/forelesninger/in2000.2026.02.11.teamarbeid-og-smidig.pdf

Mavuduru, A. (2021, 24. november) *How to perform anomaly detection with the Isolation Forest algorithm*. Towards Data Science. https://towardsdatascience.com/how-to-perform-anomaly-detection-with-the-isolation-forest-algorithm-e8c8372520bc/

Miro. (u.å). *Kanban vs. Scrum boards: 11 differences to help you choose the best tool.* Miro. https://miro.com/blog/scrum-kanban-boards-differences/

Sjøberg, S. & Bergersen, G. (2026). *Forskningsmetoder / Evaluering av IT-systemer* [Lysarkpresentasjon]. https://www.uio.no/studier/emner/matnat/ifi/IN2000/v26/forelesninger/in2000.2026.3.4.metode.pdf

Sommerville, I. (2020) *Engineering Software products: An Introduction to Modern Software Engineering.* Pearson.

Statens kartverk (u.å.a) *Adresse REST-API.* data.norge.no https://data.norge.no/nb/data-services/88458c1b-ee1a-364b-81e7-f56cbe305d38/adresse-rest-api

Statens kartverk (u.å.b) *Åpent stedsnavn API fra Kartverket.* data.norge.no https://data.norge.no/nn/data-services/f5976088-d214-3fd9-8b9e-cd3a68416469/apent-stedsnavn-api-fra-kartverket

UUtilsynet. (2026). *1.4.3 Kontrast (minimum) (Nivå AA)*. UUtilsynet. https://www.uutilsynet.no/wcag-standarden/143-kontrast-minimum-niva-aa/95#heading-7766

WebAIM. (2026). *Contrast Checker.* WebAIM. https://webaim.org/resources/contrastchecker/

